

Exploratory Data Analysis on the [Hourly Wage Data] Dataset

Report

Created By: Ashwini Ghamandi

Introduction:

The dataset chosen for this analysis is **the Hourly Wage Data** dataset. This dataset contains information about employees hourly pay along with several factors related to their work roles and demographics. Each row in the dataset represents a different employee, while each column provides specific details about that employee, such as their position, age group, experience level, and hourly wage.

The purpose of this dataset is to help explore and understand patterns in employees' wages. By analysing the data, it is possible to identify how factors such as work experience, age, and job position may influence how the employees earn per hour.

The dataset includes four main variables:

- **position:** A variable representing the employee's job role or position category. Coded values are 0 and 1.
- **agerange:** A variable that groups employees into age ranges instead of using exact age. Makes it easier to compare wage pattern across different age groups.
- **yrsscale:** A variable that represent employees experience level. Higher value generally indicates more work experience.
- **hourwage:** It shows employees hour wages. This is main variable being analysed in the dataset.

Since dataset contains both numerical and categorical data, it is suitable for many different types of analysis. Techniques such as descriptive statistics, correlation analysis, and data visualisation can be applied to gain insights from the data. Visualisations like histograms, scatter plots, box plot and bar charts can help show distributions and relationships between variables more clearly.

Overall, the **Hourly Wage Data** dataset provides useful insights into employee wages and work trends. Analysing this dataset can help reveal factors that may influence wages and improve understanding of earnings across different groups of employees.

Data Cleaning:

Before analysing the **Hourly Wage Data** dataset, several data cleaning steps were carried out to improve the quality and reliability of the data. Data cleaning is an important process because raw dataset often contains missing values, formatting issues, or inconsistent data that can affect the accuracy of the analysis.

First step data types of each column were reviewed. Columns such as hourwage that should contain numerical values were initially stored as text (strings). The hourwage column was converted from string data type to floating point numeric type so statistical calculations and visualisations could be performed correctly.

Next, step involved checking the dataset for missing or blank values. Some records contain empty spaces or missing entries particularly in column hourwage. These values were identified and filled with the mean value of the column.

The dataset was also checked for duplicate records to ensure that no employee information was repeated. Any duplicate rows found were removed to avoid biased results in the analysis.

Finally cleaned dataset was organised into structured format. These cleaning steps ensured that the dataset was accurate, consistent and suitable for further statistical analysis and exploration.

Missing Data:

The **Hourly Wage Data** dataset was carefully checked for missing data to make sure the analysis would be accurate and trustworthy. Missing data refers to values that are empty, blank, or were not recorded properly in the dataset. If missing data values are not handled correctly, they can lead to inaccurate calculations, misleading trends, and unreliable visualisation.

As a part of cleaning process, each column in the dataset was examined to identify missing values. Python functions such as `isnull ()` and `sum ()` were used to detect empty entries in the dataset.

Particular attention was given to the `hourwage` column because it represents the main variable being analysed in the dataset. Missing values could significantly affect calculations such as average, minimums, maximums and other statistical measures. To maintain the quality of the analysis, rows containing incomplete or invalid wage were filled with mean of the column.

Overall handling the missing data improved the accuracy, consistency, and reliability of the dataset. Cleaning these issues ensured that the dataset was suitable for further statistical analysis and data visualisation

Data Stories and visualisations:

Data stories:

The EDA process helped create several data stories and visualisations that explained patterns and relationships within **Hourly Wage Data** dataset. By using graphs and charts, the analysis transformed raw numerical data into meaningful insights that were easier to understand and interpret.

One of the key stories discovered during the analysis was the relationship between experience and hourly wage. The Years scale vs hourly wage graph showed a clearer pattern between experience and pay. As the years of experience scale increased, hourly wages generally appeared to increase as well. This suggested that employees with greater experience levels were more likely to earn higher hourly wages compared to less experienced workers.

Another important story involved job position and wage variation. In the position via Hourly graphs, employees from both position groups earned a wide range of wages, suggesting that job position alone did not fully determine employee pay. While some employees earned lower hourly wages, others received significantly higher pay, indicating variation within each position category.

The visualisation also revealed patterns related to age groups and hourly wages. The Age Range vs Hourly Wage graph revealed that employees across all age groups experience differences in hourly earnings. However, some age groups

appeared to have slightly higher wage concentrations which may suggest the influence of seniority, experience or career progression on employee earnings.

The average wage comparisons showed only moderate differences between position groups, while experience categories display a gradual increase in wages as experience level increased.

The visualisations helped transform raw numerical data into meaningful insights. They showed that hourly wages were influenced by multiple factors, particularly experience level, while position and age also contributed to wage variation across employees.

The distribution of hourly wages was analysed across yrsscale, agerange and position using different graphs. The result indicates a clear positive relationship between yrsscale and across both position groups, where wages generally increased as yrsscale increased from 0 to 6. In both positions, lower yrsscale levels showed a concentration of wages within the upper range.

When examining position specific trends, position 0 exhibited a more stable wage distribution, particularly at higher yrsscale values, where wages were consistently clustered between 20 and 30 units. In contrast position 1 demonstrated greater variability in wages across all yrsscale levels, although an overall upward trend with increase experience was still observed.

Age range further influenced wage distribution, with younger employees (age range 1) predominantly appearing in lower wage categories and lower yrsscale groups. Conversely, older employees (age range 3) were more frequently associated with higher wages and higher yrsscale values. Overall, the analysis suggested that years of experience was the strongest determinant of hourly wage, while age and position contributed secondary but noticeable effects on wage variability and distribution.

Several visualisation techniques were used during the EDA process

- Scatter plot: were used to identify relationships between variables such as experience, position, age range and hourly wage.
- Bar Charts compared to average wages and category counts across employee groups.
- Histograms: displays the distribution of hourly wages and highlighted common wage ranges.

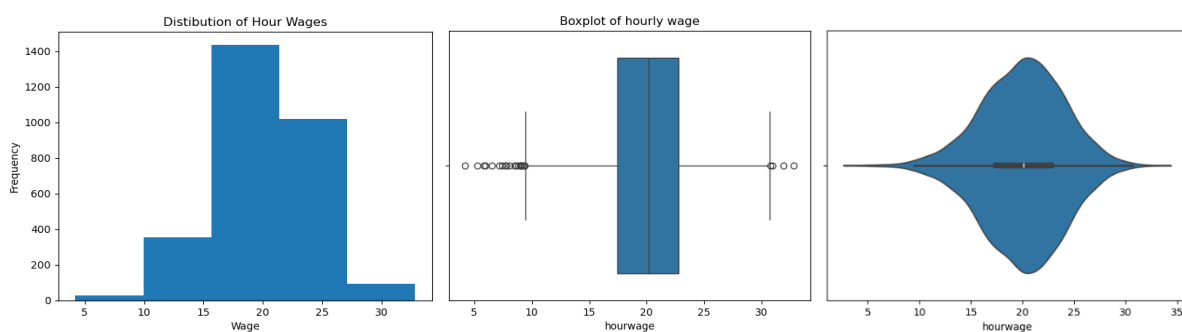
- Box plots showed the spread of wages, median values and possible outliers in the dataset
- Heat maps were used to visualisation of correlations between numerical variables in the datasets. identified strong and weak relationships between variables.
- violin plots were used to examine the distribution and density of hourly wages.

These visualisations made the findings more engaging and easier to interpret compared to analysing tables of raw numbers alone.

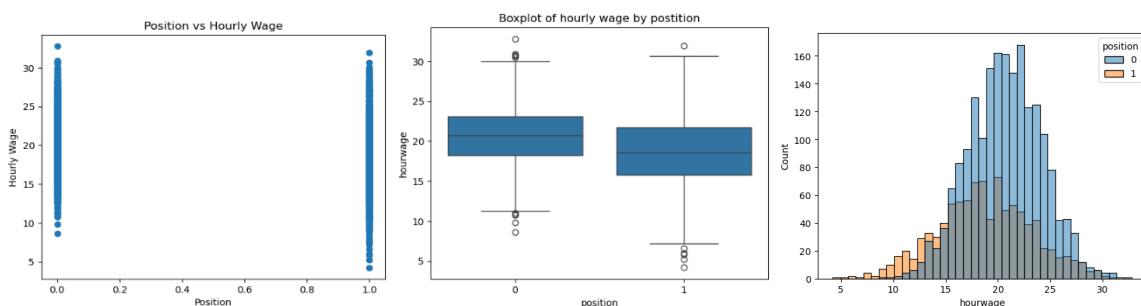
Overall, the data stories and visualisations provided valuable insight into employee compensation patterns and helped explained how factors such as experience, position and age range may have influenced hourly wages within the dataset

Visualisations:

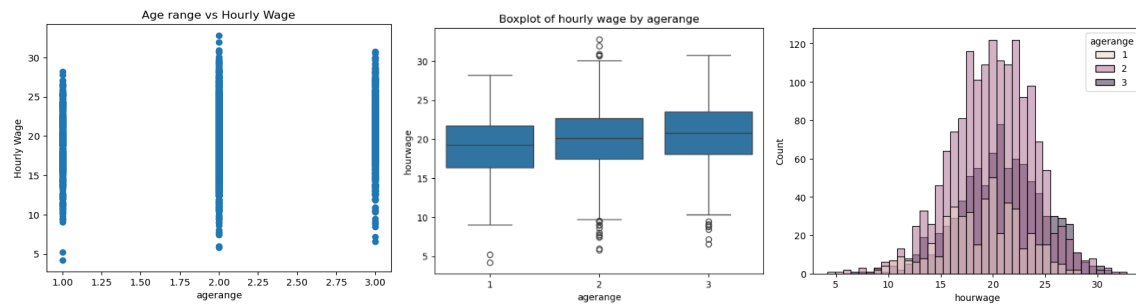
Hour Wages:



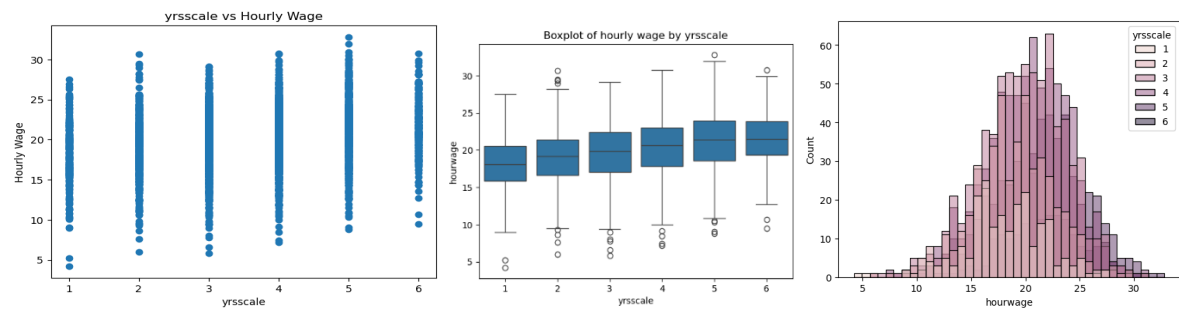
Hour Wages by Position:



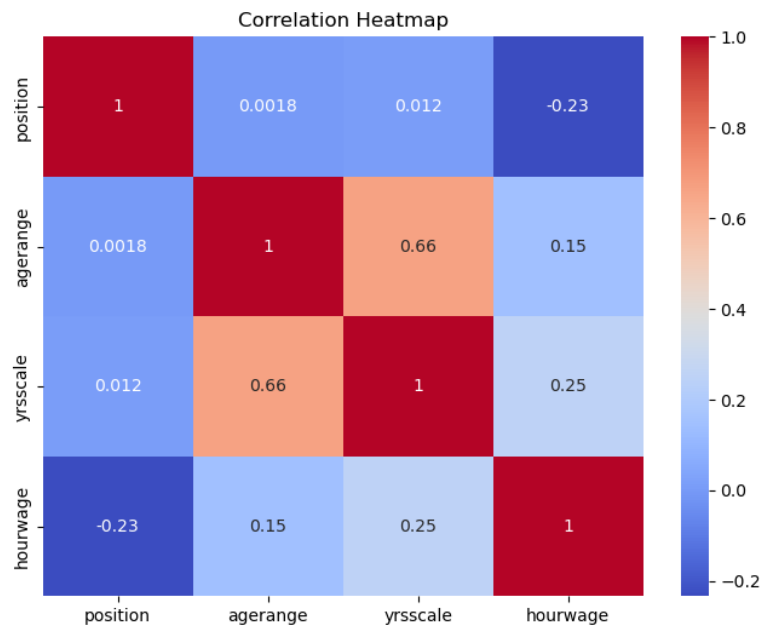
Hour Wages by Age Range:



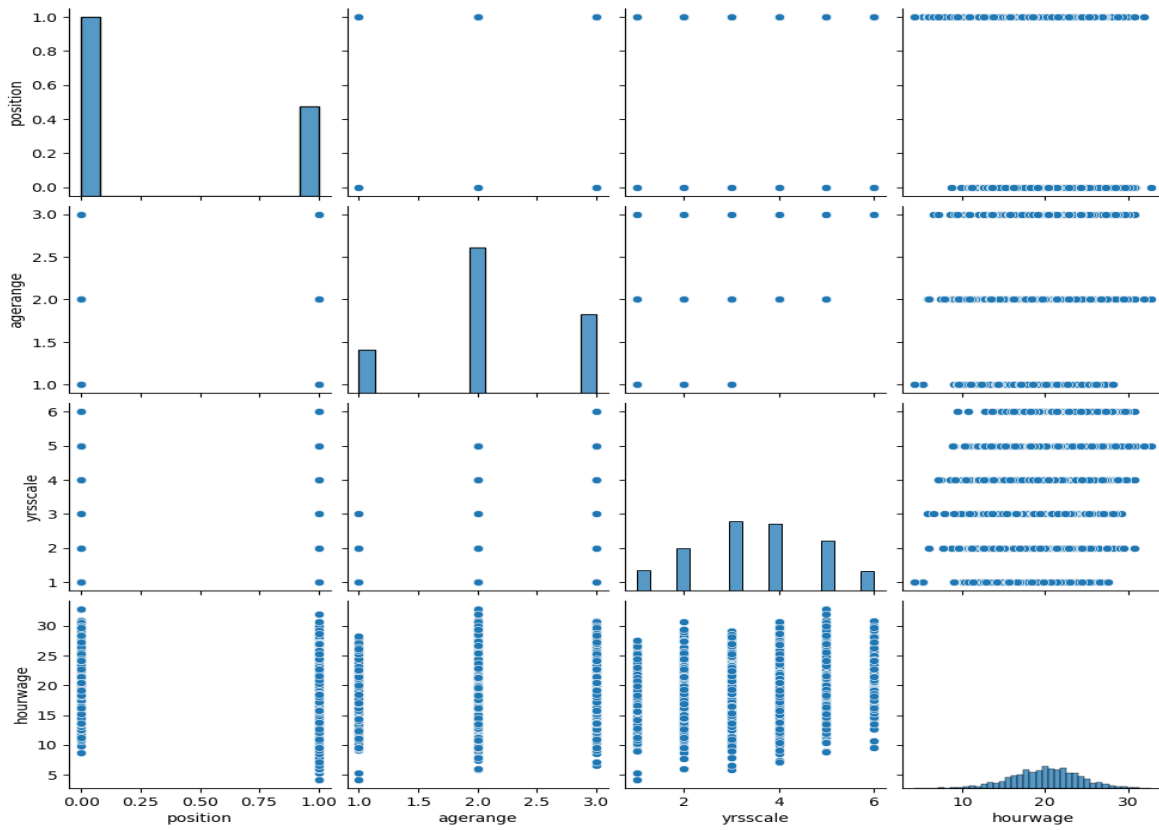
Hour Wages by yrs scale:



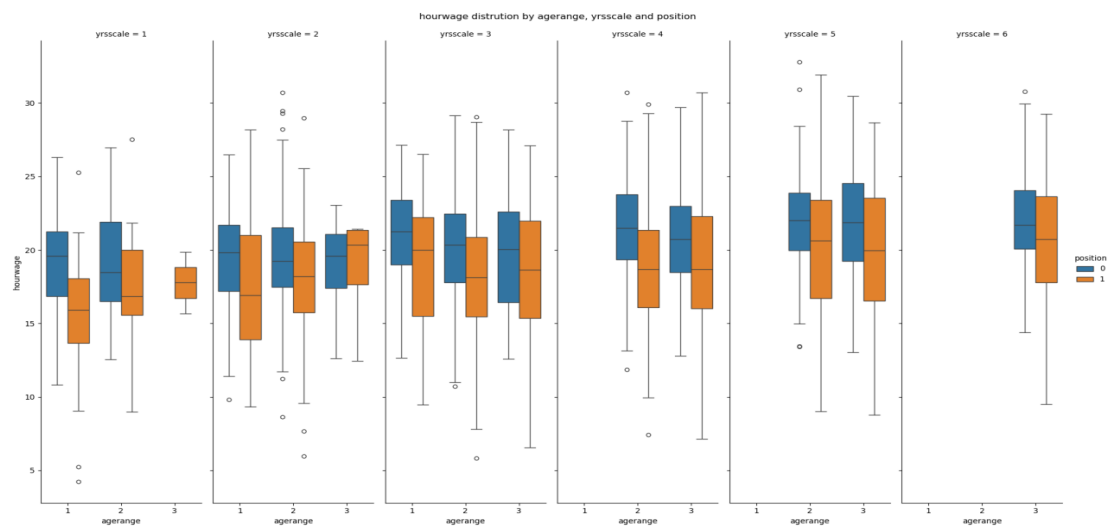
Correlation Heatmap:



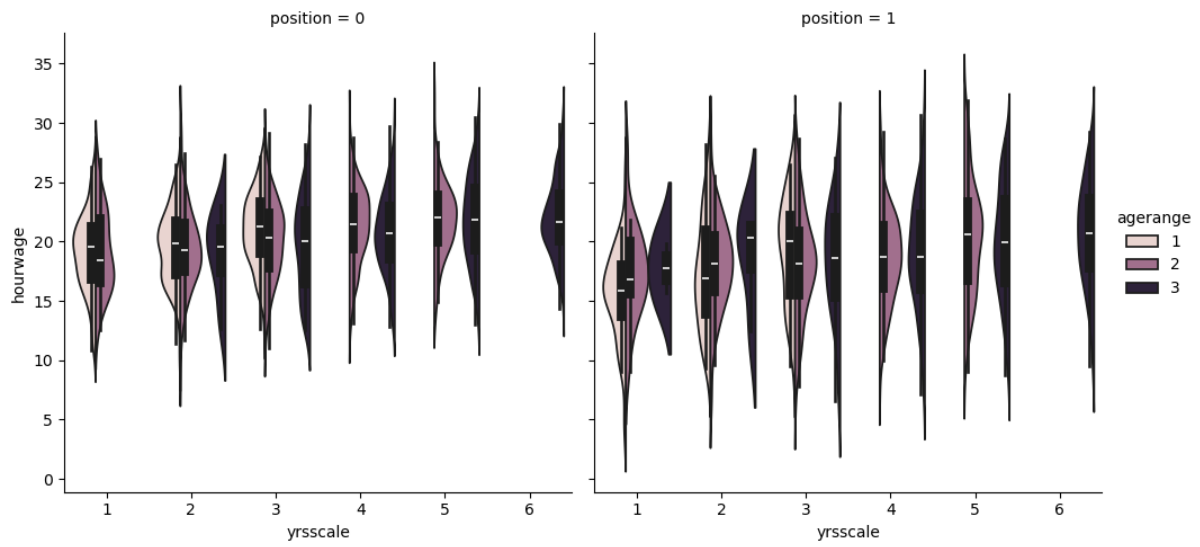
All Relationships:



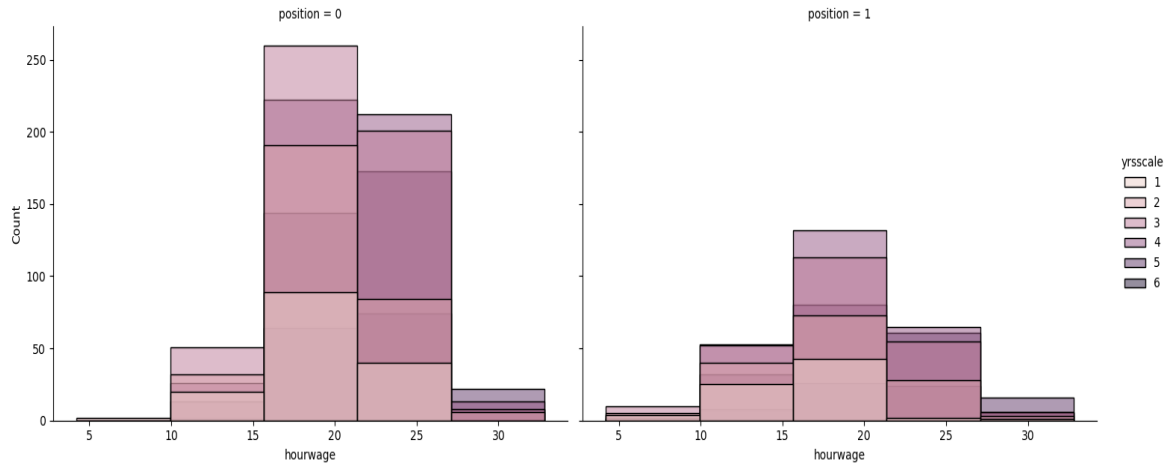
Relations between position, age range, yrsscale, and hours wage



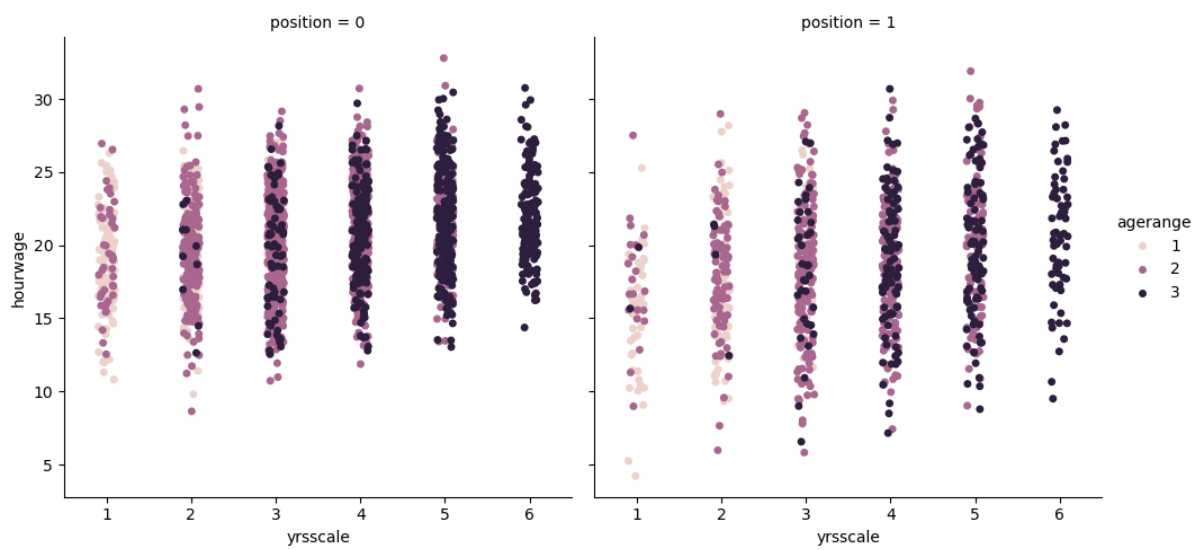
hourwage distrution by agerange, yrsscale and position



hourwage distrution by yrsscale and position



hourwage distrution by agerange, yrsscale and position





Appendix:

Exploratory data Analysis in Python

Imported all needed python libraries like pandas, numpy matplotlib.pyplot

Loaded hourlywagedata.csv file using pd.read_csv

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

hourlywagedata_df = pd.read_csv('hourlywagedata.csv',
                                names=['position', 'agerange', 'yrsscale', 'hourwage'],
                                skiprows=1,
                                low_memory=False)
type(hourlywagedata_df)
pandas.DataFrame
```

Data Exploration

Get the datatype of dataframe

```
hourlywagedata_df.dtypes
position    int64
agerange   int64
yrsscale    int64
hourwage    str
dtype: object
```

Converted hourwage column into numerical and then converted it into float

```
hourlywagedata_df['hourwage'] = pd.to_numeric(hourlywagedata_df['hourwage'], errors='coerce')
hourlywagedata_df['hourwage'] = hourlywagedata_df['hourwage'].astype(float)
```

Inspect the data. get top 2 records using head() and transpose it for better visualisation

```
hourlywagedata_df.head(2).T
```

	0	1
position	1.000000	0.000000
agerange	1.000000	1.000000
yrsscale	2.000000	2.000000
hourwage	13.736234	16.440731

Get the shape of the data frame , number of rows and columns

```
hourlywagedata_df.shape
```

(3000, 4)

Get the description of data like count of the records, mean, std, mean, 25%, 50% , 75% and max of the columns in the dataset

```
hourlywagedata_df.describe()
```

	position	agerange	yrsscale	hourwage
count	3000.000000	3000.000000	3000.000000	2911.000000
mean	0.333333	2.136000	3.527333	20.015860

	position	agerange	yrsscale	hourwage
std	0.471483	0.661551	1.349022	4.003089
min	0.000000	1.000000	1.000000	4.215732
25%	0.000000	2.000000	3.000000	17.415044
50%	0.000000	2.000000	4.000000	20.180008
75%	1.000000	3.000000	5.000000	22.784393
max	1.000000	3.000000	6.000000	32.803854

Check for any null values in the dataset and sum them for each column

```
hourlywagedata_df.isnull().sum()
position    0
agerange    0
yrsscale    0
hourwage    89
dtype: int64
```

Count the number of unic distinct columns

```
hourlywagedata_df.index.nunique()
3000
```

Get more information about dataset

```
hourlywagedata_df.info()
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 4 columns):
#   Column   Non-Null Count  Dtype
---  -
0   position 3000 non-null   int64
1   agerange 3000 non-null   int64
2   yrsscale 3000 non-null   int64
3   hourwage 2911 non-null   float64
dtypes: float64(1), int64(3)
memory usage: 93.9 KB
```

Get two random samples from datasets

```
hourlywagedata_df.sample(2, random_state=1).T
```

	1957	2087
position	0.00000	0.000000
agerange	2.00000	2.000000
yrsscale	4.00000	3.000000
hourwage	19.13787	22.761852

Data Cleaning

fill missing values

```
hourlywagedata_df['hourwage'] =  
hourlywagedata_df['hourwage'].fillna(hourlywagedata_df['hourwage'].mean())
```

Check if dataset contains duplicatates

```
hourlywagedata_df.duplicated().sum()  
np.int64(69)
```

Remove duplicates

```
hourlywagedata_df.drop_duplicates(inplace=True)
```

Check final cleaned data with .info(), .isnull().sum(), .head() and .shape

```
hourlywagedata_df.info()  
<class 'pandas.DataFrame'>
```

```

Index: 2931 entries, 0 to 2999
Data columns (total 4 columns):
#   Column   Non-Null Count  Dtype
---  -
0    position 2931 non-null   int64
1    agerange 2931 non-null   int64
2    yrsscale 2931 non-null   int64
3    hourwage 2931 non-null   float64
dtypes: float64(1), int64(3)
memory usage: 114.5 KB
hourlywagedata_df.isnull().sum()
position    0
agerange    0
yrsscale    0
hourwage    0
dtype: int64
hourlywagedata_df.head()

```

	position	agerange	yrsscale	hourwage
0	1	1	2	13.736234
1	0	1	2	16.440731
2	0	1	3	21.389108
3	1	1	1	11.377187
4	0	1	3	21.560778

```

hourlywagedata_df.shape
(2931, 4)

```

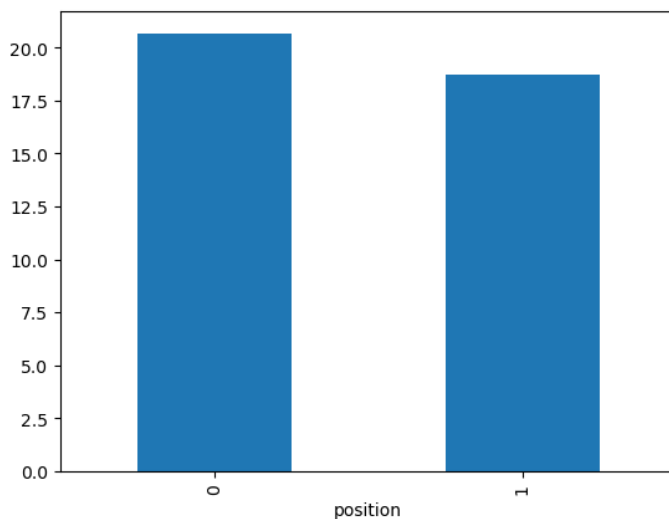
Preparing Data For EDA

Using groupby method in pandas to group by position, agerange and yrsscale columns and then getting mean using hourwage column

```
hourlywagedata_df.groupby('position')['hourwage'].mean()
position
0    20.672668
1    18.698203
Name: hourwage, dtype: float64
```

Bar Plot for displaying position categories mean hourwage

```
hourlywagedata_df.groupby('position')['hourwage'].mean().plot.bar()
<Axes: xlabel='position'>
```

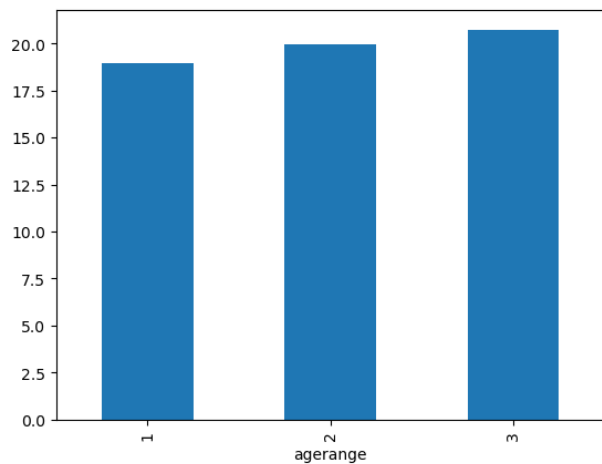


Using groupby method in pandas to group by agerange column and then getting mean using hourwage column

```
hourlywagedata_df.groupby('agerange')['hourwage'].mean()
agerange
1    18.926365
2    19.949784
3    20.730317
Name: hourwage, dtype: float64
```

Bar Plot for displaying agerange categories mean hourwage

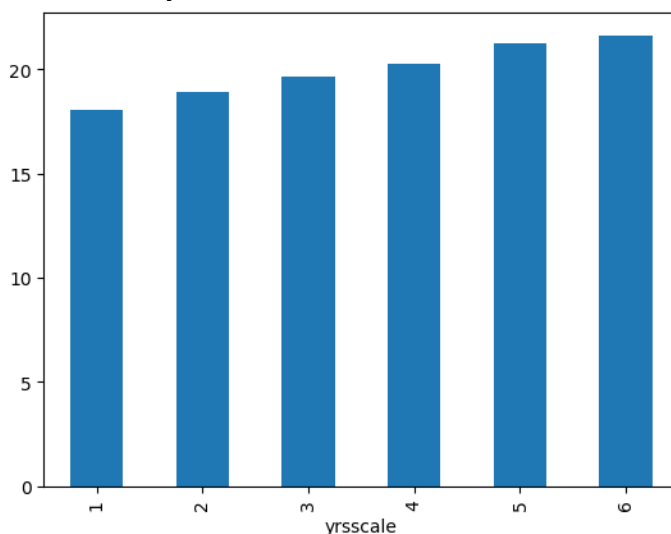
```
hourlywagedata_df.groupby('agerange')['hourwage'].mean().plot.bar()
<Axes: xlabel='agerange'>
```



Using groupby method in pandas to group by yrsscale column and then getting mean using hourwage column

Bar Plot for displaying yrsscale categories mean hourwage

```
hourlywagedata_df.groupby('yrsscale')['hourwage'].mean()
yrsscale
1    18.068036
2    18.926420
3    19.663429
4    20.286148
5    21.250276
6    21.626483
Name: hourwage, dtype: float64
hourlywagedata_df.groupby('yrsscale')['hourwage'].mean().plot.bar()
<Axes: xlabel='yrsscale'>
```



Sort data

```
sorted_data = hourlywagedata_df.sort_values("hourwage")
print(sorted_data.head())
```

```
   position  agerange  yrsscale  hourwage
318        1         1         1  4.215732
267        1         1         1  5.236800
1395       1         2         3  5.819950
1971       1         2         2  5.978098
2667       1         3         3  6.560689
```

Basic Descriptive Analysis

Describes the given columns of dataset

```
hourlywagedata_df.describe()
```

	position	agerange	yrsscale	hourwage
count	2931.000000	2931.000000	2931.000000	2931.000000
mean	0.332651	2.134766	3.525418	20.015860
std	0.471243	0.662441	1.351886	3.989404
min	0.000000	1.000000	1.000000	4.215732
25%	0.000000	2.000000	3.000000	17.433059
50%	0.000000	2.000000	4.000000	20.140169
75%	1.000000	3.000000	5.000000	22.763056
max	1.000000	3.000000	6.000000	32.803854

calculates the mode of given columns of dataset

```
hourlywagedata_df['hourwage'].mode()  
0    20.01586  
Name: hourwage, dtype: float64
```

calculates the mean of given columns of dataset

```
hourlywagedata_df['hourwage'].mean()
```

calculates the median of given columns of dataset

```
hourlywagedata_df['hourwage'].median()  
np.float64(20.1401686129256)
```

calculates the standard deviation of given columns of dataset

```
hourlywagedata_df['hourwage'].std()  
np.float64(3.989403522144717)
```

calculates the variance of given columns of dataset

```
hourlywagedata_df['hourwage'].var()  
np.float64(15.915340462500675)
```

calculates the min of given columns of dataset

```
hourlywagedata_df['hourwage'].min()  
np.float64(4.2157322423472)
```

calculates the max of given columns of dataset

```
hourlywagedata_df['hourwage'].max()  
np.float64(32.8038537410009)
```

Describes the given columns of dataset

```
totvars=['position','agerange','yrsscale','hourwage']  
hourlywagedata_df[totvars].describe()
```

	position	agerange	yrsscale	hourwage
count	2931.000000	2931.000000	2931.000000	2931.000000
mean	0.332651	2.134766	3.525418	20.015860
std	0.471243	0.662441	1.351886	3.989404
min	0.000000	1.000000	1.000000	4.215732
25%	0.000000	2.000000	3.000000	17.433059
50%	0.000000	2.000000	4.000000	20.140169
75%	1.000000	3.000000	5.000000	22.763056
max	1.000000	3.000000	6.000000	32.803854

Find values at specific points in a dataset after sorting.

```
hourlywagedata_df[totvars].quantile(np.arange(0.0,1.1,0.1))
```

	position	agerange	yrsscale	hourwage
0.0	0.0	1.0	1.0	4.215732
0.1	0.0	1.0	2.0	14.942104
0.2	0.0	2.0	2.0	16.696164
0.3	0.0	2.0	3.0	17.964337
0.4	0.0	2.0	3.0	19.224012

	position	agerange	yrsscale	hourwage
0.5	0.0	2.0	4.0	20.140169
0.6	0.0	2.0	4.0	21.176999
0.7	1.0	2.0	4.0	22.161562
0.8	1.0	3.0	5.0	23.406450
0.9	1.0	3.0	5.0	24.985920
1.0	1.0	3.0	6.0	32.803854

Skewness measures how asymmetrical a data distribution is.

```
hourlywagedata_df['hourwage'].skew(numeric_only=True)
np.float64(-0.22180131342168588)
```

How much data is concentrated at the end of the dataset as compare to center

```
hourlywagedata_df['hourwage'].kurtosis(numeric_only=True)
np.float64(0.1424249934761619)
```

unique records in the dataset

```
hourlywagedata_df['hourwage'].nunique()
2912
```

counts nuber of occurance of each unique value

```
hourlywagedata_df['hourwage'].value_counts()
hourwage
20.015860    20
13.736234     1
16.440731     1
21.389108     1
11.377187     1
```

```
..
20.419885  1
20.441482  1
18.450234  1
19.144460  1
13.638466  1
Name: count, Length: 2912, dtype: int64
```

Correlation Analysis

```
hourlywagedata_df.corr(numeric_only=True)
```

	position	agerange	yrsscale	hourwage
position	1.000000	0.001752	0.012170	-0.233231
agerange	0.001752	1.000000	0.664823	0.146408
yrsscale	0.012170	0.664823	1.000000	0.249102
hourwage	-0.233231	0.146408	0.249102	1.000000

Visualisation

Histogram (Distribution of data):
understanding spread of numeric data

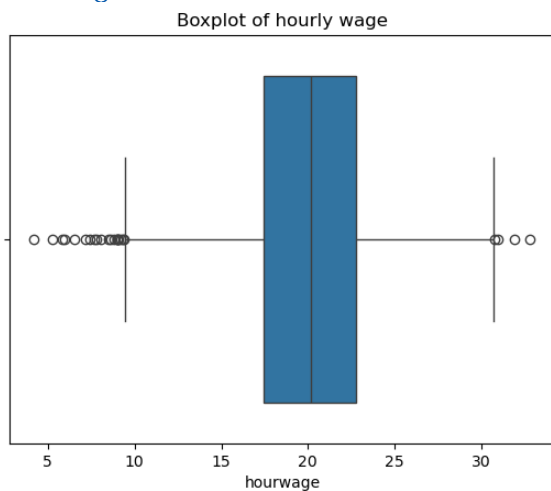
```
plt.hist(hourlywagedata_df['hourwage'], bins=5)
plt.title("Distibution of Hour Wages")
```

```
plt.xlabel("Wage")
plt.ylabel("Frequency")
plt.show()
```



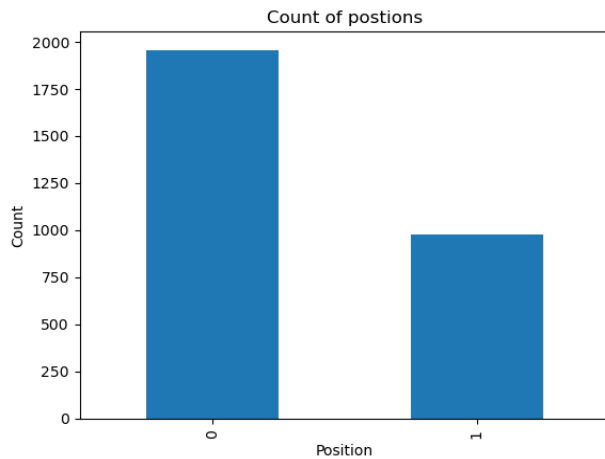
Box Plot (Outliers + spread)

```
sns.boxplot(data=hourlywagedata_df,x=hourlywagedata_df['hourwage'])
plt.title("Boxplot of hourly wage")
plt.show()
```

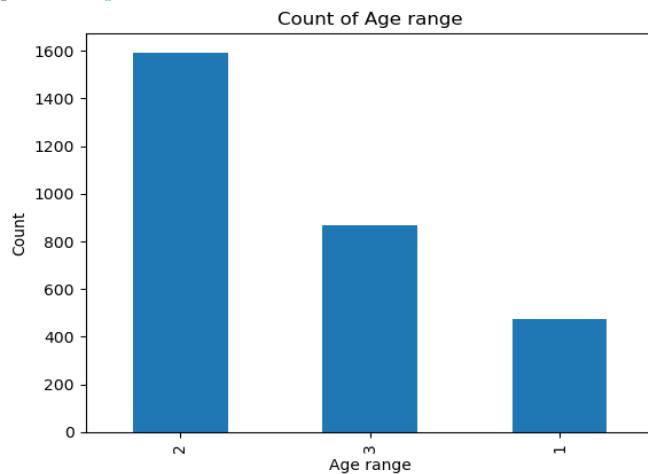


Bar chart (categorical data)

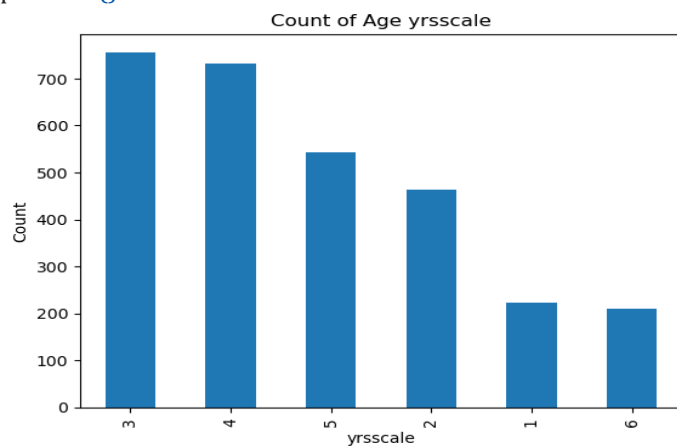
```
hourlywagedata_df['position'].value_counts().plot(kind='bar')
plt.title("Count of postions")
plt.xlabel("Position")
plt.ylabel("Count")
plt.show()
```



```
hourlywagedata_df['agerange'].value_counts().plot(kind='bar')
plt.title("Count of Age range")
plt.xlabel("Age range")
plt.ylabel("Count")
plt.show()
```

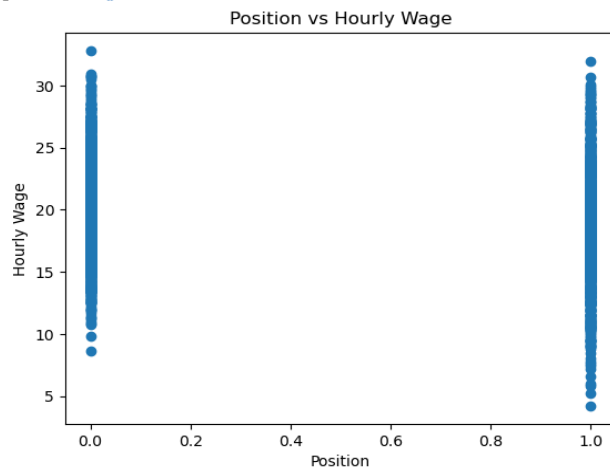


```
hourlywagedata_df['yrsscale'].value_counts().plot(kind='bar')
plt.title("Count of Age yrsscale")
plt.xlabel("yrsscale")
plt.ylabel("Count")
plt.show()
```



Scatter Plot (Relationship Between Variables)

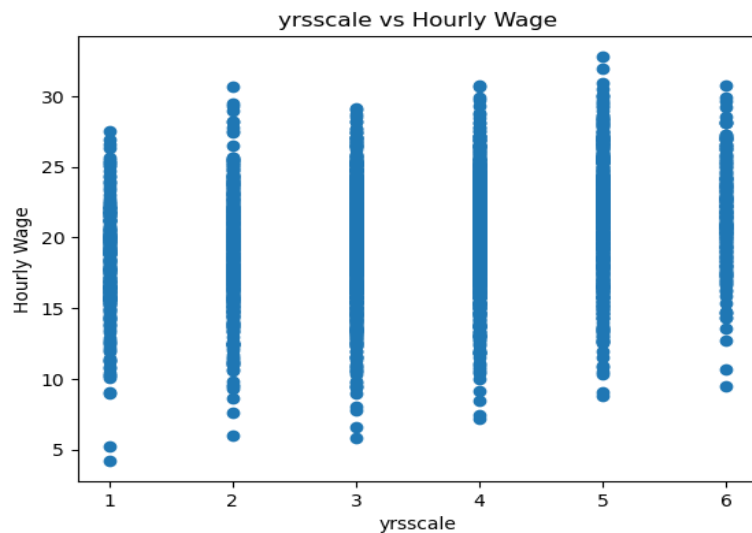
```
plt.scatter(hourlywagedata_df['position'],hourlywagedata_df['hourwage'])
plt.title("Position vs Hourly Wage")
plt.xlabel("Position")
plt.ylabel("Hourly Wage")
plt.show()
```



```
plt.scatter(hourlywagedata_df['agerange'],hourlywagedata_df['hourwage'])
plt.title("Age range vs Hourly Wage")
plt.xlabel("agerange")
plt.ylabel("Hourly Wage")
plt.show()
```

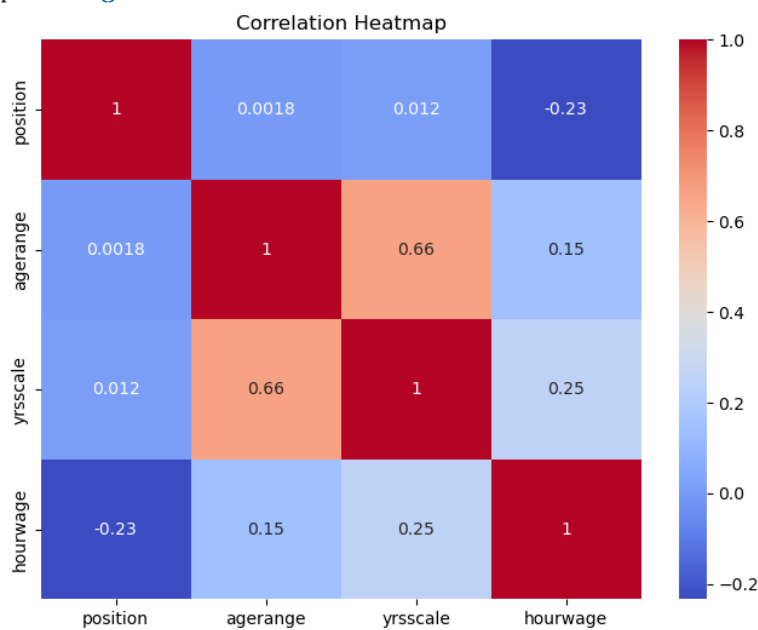


```
plt.scatter(hourlywagedata_df['yrsscale'],hourlywagedata_df['hourwage'])
plt.title("yrsscale vs Hourly Wage")
plt.xlabel("yrsscale")
plt.ylabel("Hourly Wage")
plt.show()
```



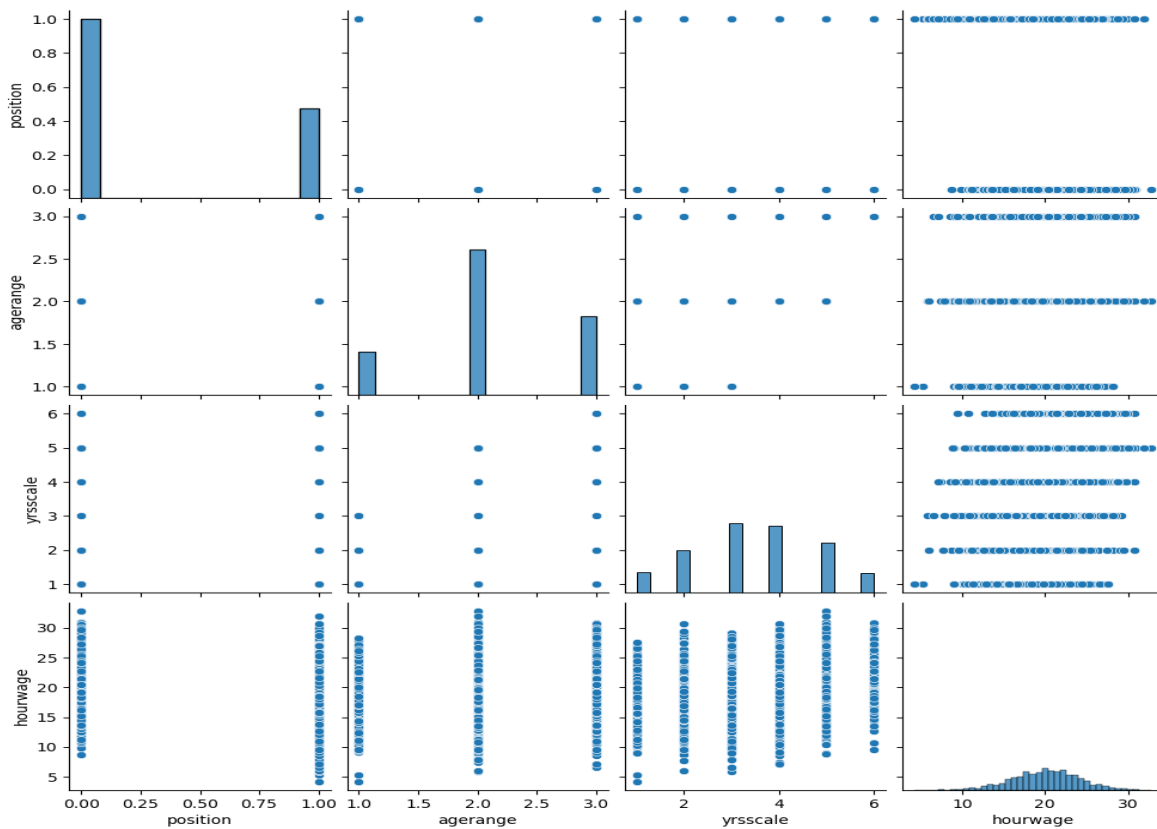
Heatmap (Correlation Matrix)

```
plt.figure(figsize=(8,6))
sns.heatmap(hourlywagedata_df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



Pairplot (All Relationships)

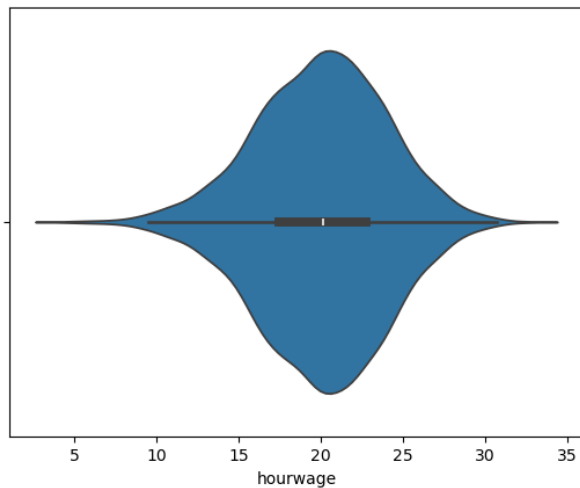
```
sns.pairplot(hourlywagedata_df)
plt.show()
```



Violin plot Distribution + density()

```
sns.violinplot(x=hourlywagedata_df['hourwage'])
plt.show
```

```
<function matplotlib.pyplot.show(close=None, block=None)>
```



```
pd.crosstab(index=hourlywagedata_df['yrsscale'], columns=hourlywagedata_df['position'])
```

position	0	1
----------	---	---

yrsscale

1	149	75
---	-----	----

2	315	149
---	-----	-----

3	520	236
---	-----	-----

4	473	260
---	-----	-----

5	352	191
---	-----	-----

6	147	64
---	-----	----

```
pd.crosstab(index=hourlywagedata_df['yrsscale'], columns=hourlywagedata_df['agerange'])
```

agerange	1	2	3
----------	---	---	---

yrsscale

1	158	64	2
---	-----	----	---

2	188	262	14
---	-----	-----	----

3	126	527	103
---	-----	-----	-----

4	0	492	241
---	---	-----	-----

5	0	247	296
---	---	-----	-----

6	0	0	211
---	---	---	-----

```
pd.crosstab(index=hourlywagedata_df['agerange'], columns=hourlywagedata_df['position'])
```

position	0	1
----------	---	---

agerange

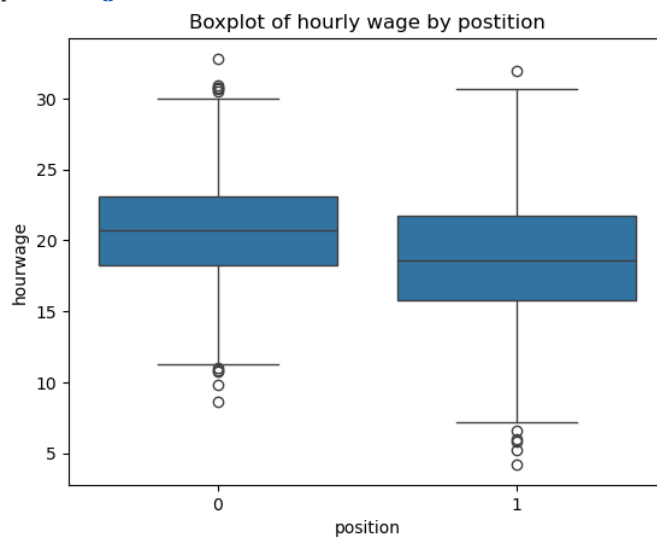
1	316	156
---	-----	-----

2	1062	530
---	------	-----

3	578	289
---	-----	-----

Box Plot

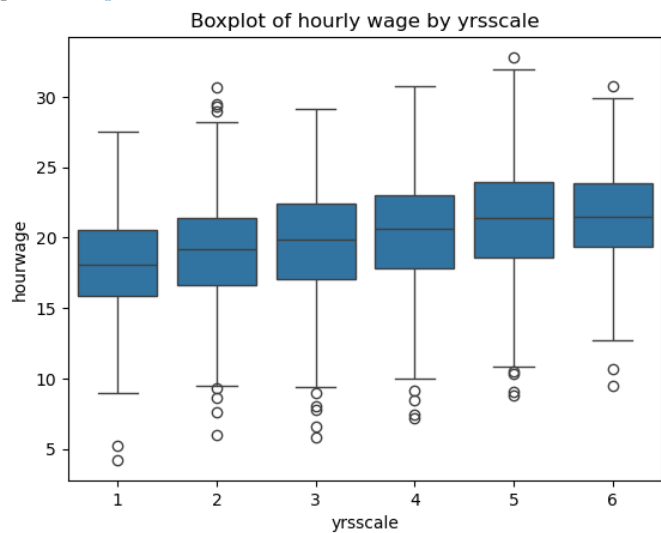
```
sns.boxplot(data=hourlywagedata_df,x=hourlywagedata_df['position'],  
y=hourlywagedata_df['hourwage'])  
plt.title("Boxplot of hourly wage by postition")  
plt.show()
```



```
sns.boxplot(data=hourlywagedata_df,x=hourlywagedata_df['agerange'],  
y=hourlywagedata_df['hourwage'])  
plt.title("Boxplot of hourly wage by agerange")  
plt.show()
```

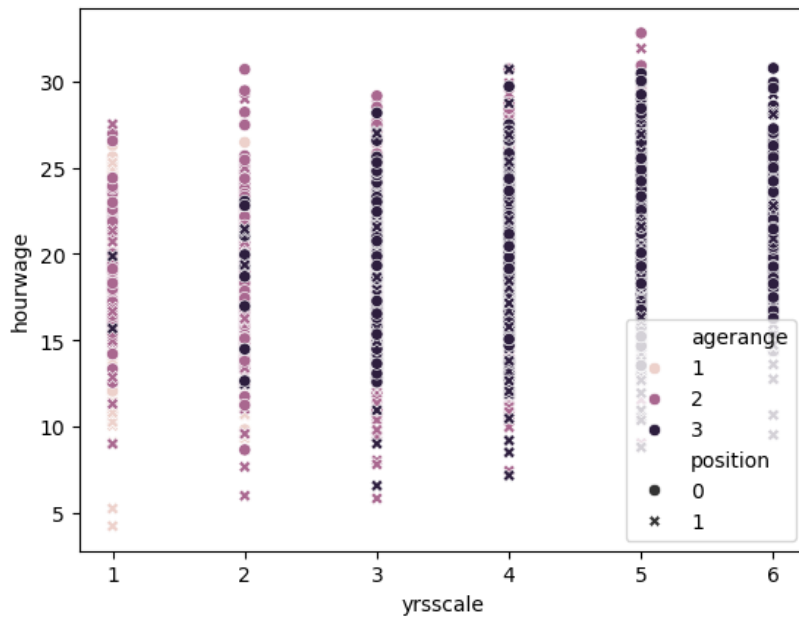


```
sns.boxplot(data=hourlywagedata_df,x=hourlywagedata_df['yrsscale'],
y=hourlywagedata_df['hourwage'])
plt.title("Boxplot of hourly wage by yrsscale")
plt.show()
```



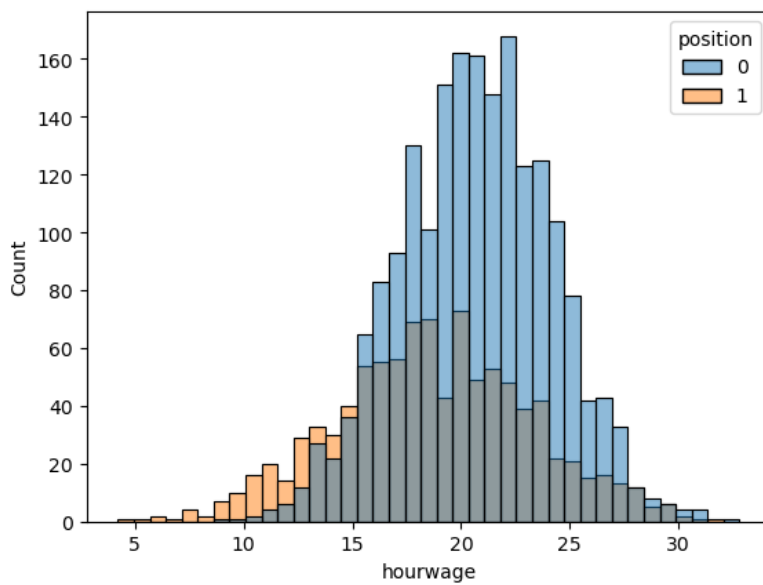
Mulivariate scatterplot

```
sns.scatterplot(data=hourlywagedata_df,
x='yrsscale',
y='hourwage',
style='position',
hue='agerange')
<Axes: xlabel='yrsscale', ylabel='hourwage'>
```



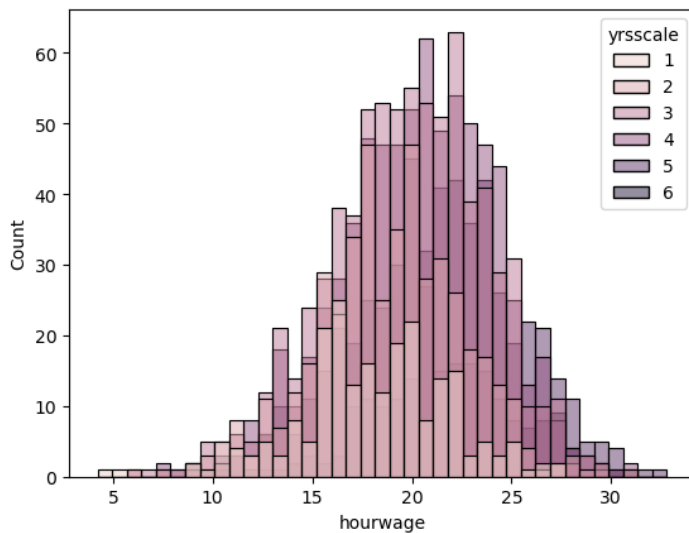
```
sns.histplot(data=hourlywagedata_df,
             x='hourwage',
             hue='position',
             multiple='layer'
)
```

<Axes: xlabel='hourwage', ylabel='Count'>

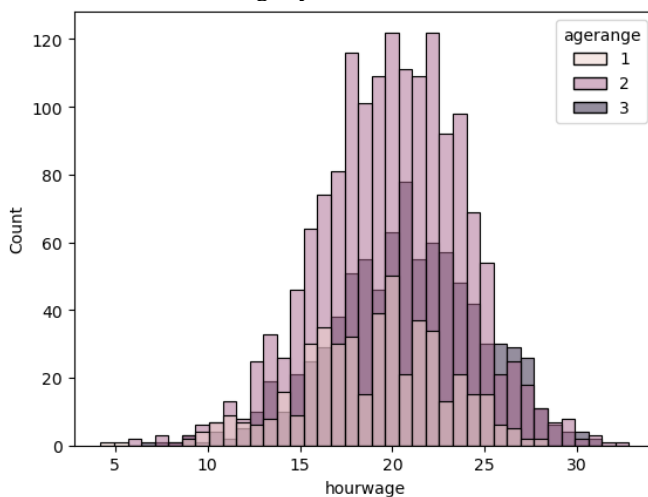


```
sns.histplot(data=hourlywagedata_df,
             x='hourwage',
             hue='yrsscale',
             multiple='layer'
)
```

<Axes: xlabel='hourwage', ylabel='Count'>

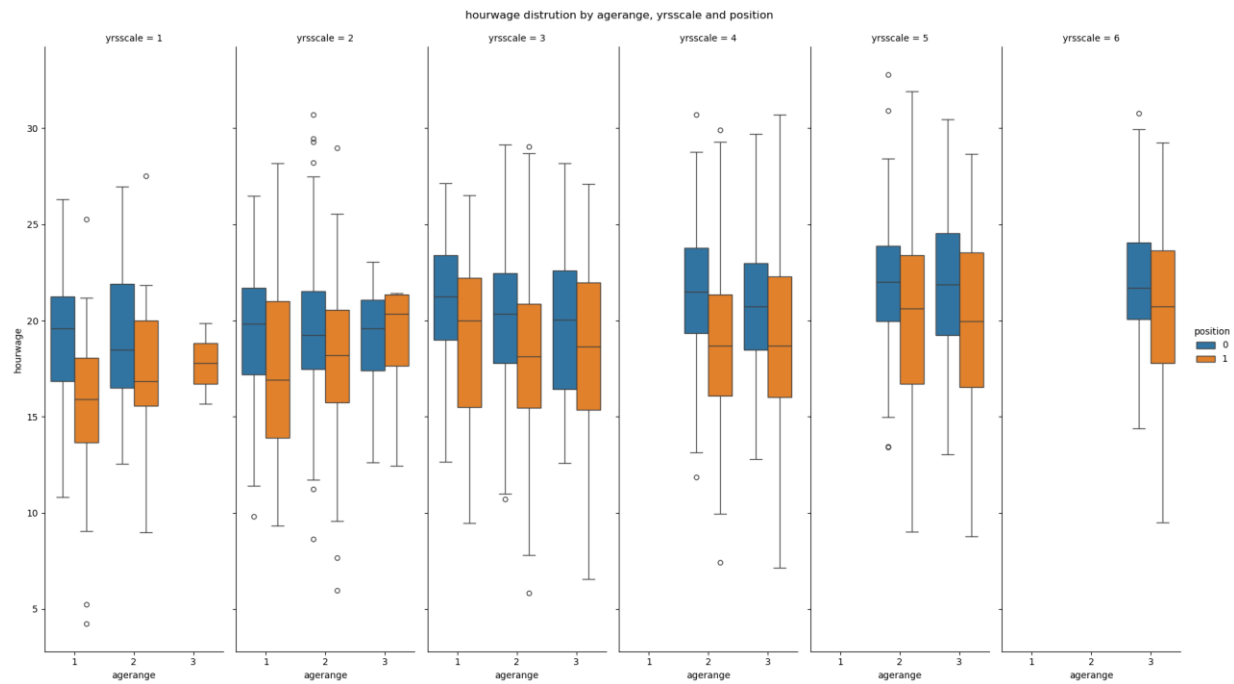


```
sns.histplot(data=hourlywagedata_df,
             x='hourwage',
             hue='agerange',
             multiple='layer'
)
<Axes: xlabel='hourwage', ylabel='Count'>
```

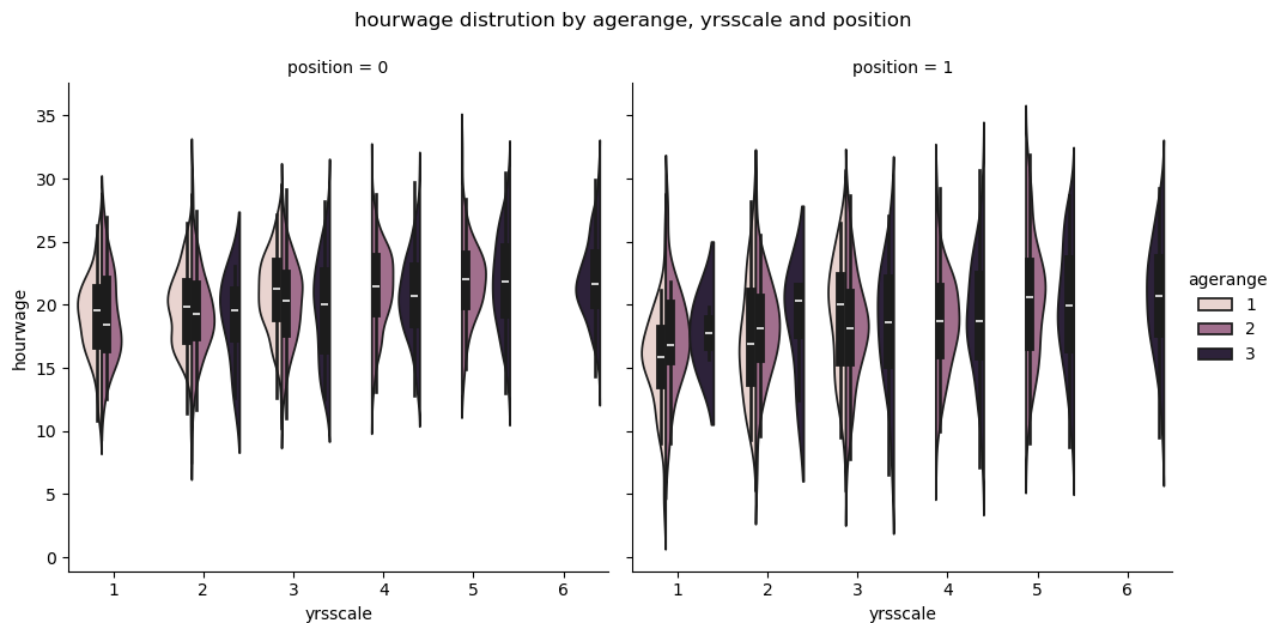


Finding relations between different variables like agerange, position, yrsscale with hourwage (distributions)

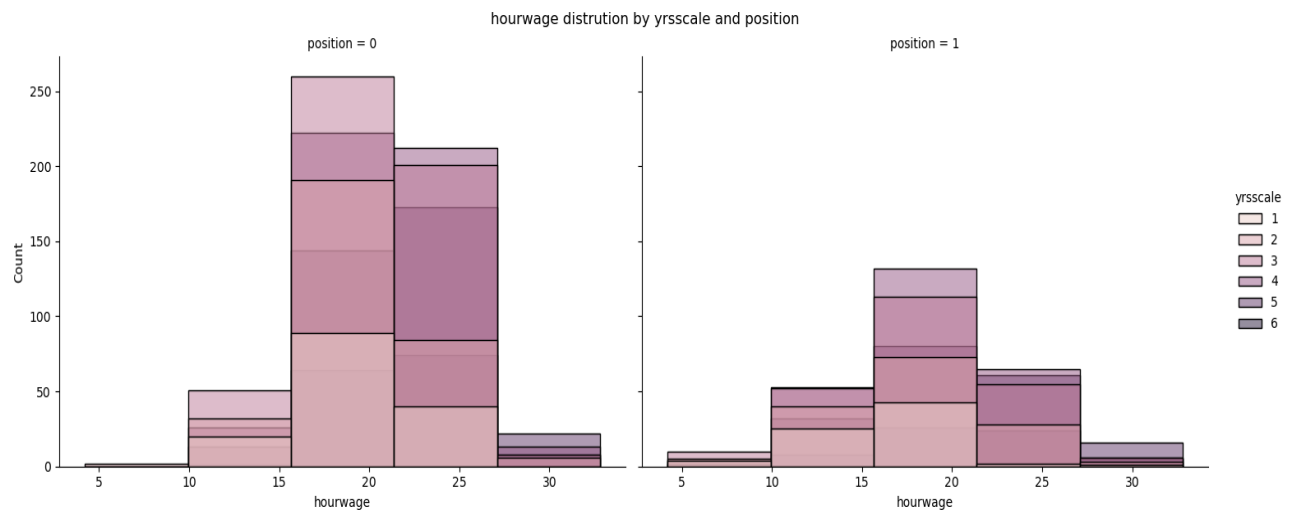
```
g = sns.catplot(x='agerange', y='hourwage', hue='position', col='yrsscale', data=hourlywagedata_df,
               kind='box', height=10, aspect=0.3)
g.fig.suptitle('hourwage distrution by agerange, yrsscale and position', y=1.02)
Text(0.5, 1.02, 'hourwage distrution by agerange, yrsscale and position')
```



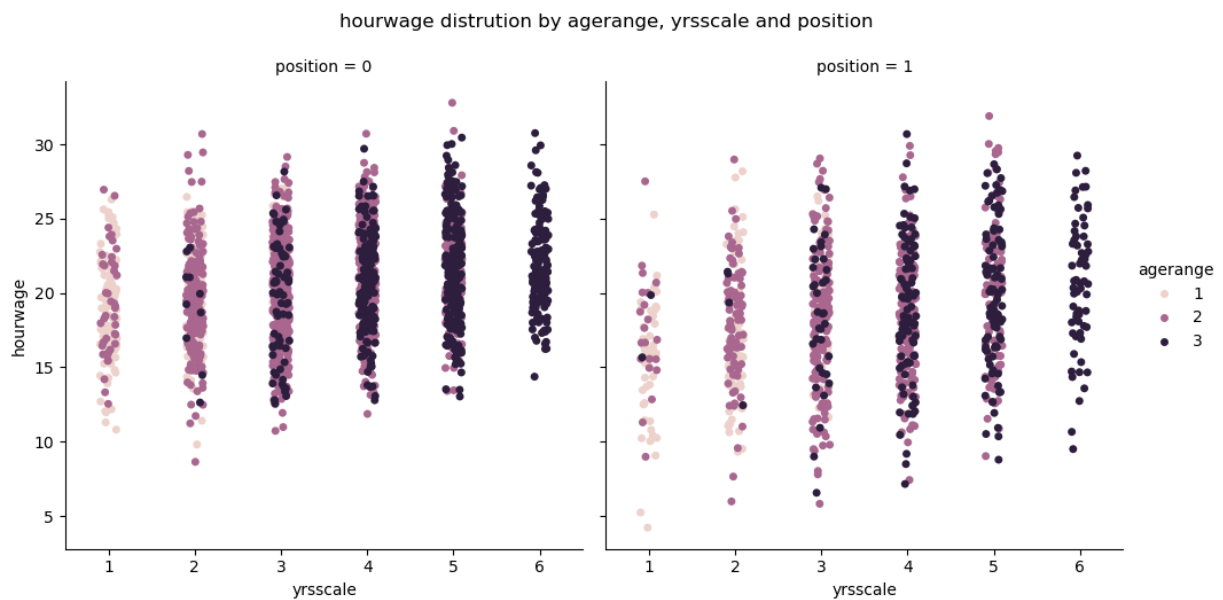
```
g = sns.catplot(x='yrsscale', y='hourwage', hue='agerange', col='position', data=hourlywagedata_df,
kind='violin', split=True, height=5, aspect=1)
g.fig.suptitle('hourwage distrution by agerange, yrsscale and position', y=1.05)
Text(0.5, 1.05, 'hourwage distrution by agerange, yrsscale and position')
```



```
g = sns.displot(x='hourwage', hue='yrsscale', col='position', data=hourlywagedata_df, kind='hist', bins=5,
height=5, aspect=1)
g.fig.suptitle('hourwage distrution by yrsscale and position', y=1.05)
Text(0.5, 1.02, 'hourwage distrution by yrsscale and position')
```



```
g = sns.catplot(x='yrsscale', y='hourwage', hue='agerange', col='position', data=hourlywagedata_df,
kind='strip', height=5, aspect=1)
g.fig.suptitle('hourwage distrution by agerange, yrsscale and position', y=1.05)
Text(0.5, 1.05, 'hourwage distrution by agerange, yrsscale and position')
```



```
g = sns.catplot(x='yrsscale', y='hourwage', hue='position', col='agerange', data=hourlywagedata_df,
kind='point', height=5, aspect=1)
g.fig.suptitle('hourwage distrution by agerange, yrsscale and position', y=1.05)
Text(0.5, 1.05, 'hourwage distrution by agerange, yrsscale and position')
```

